From the course MPC to the docking MPC

A step-by-step bridge, with every divergence named

9 October 2026

How to read this

Every step below has the same four parts. **Course.** what Section 5 of the MECHENG 724 notes does, with the notes' own equation. **Ours.** what the docking controller does. **Why it differs.** the reason for any difference, and what would go wrong without it. **Check.** which test in `matlab/tests` verifies that step of your implementation. The notes are cited by their section numbers (5.3.2, 5.4.1, ...); the design document `docking_formulation.pdf` by "DD 4.x". A worked numerical tick with small horizons runs through the whole document so you can follow the matrices by hand.

There are only four places where the docking MPC leaves the notes, and each is forced by the problem rather than by taste:

1. the plant is nonlinear, so the matrices are rebuilt every tick (*successive linearisation*);
2. the outputs have mixed units, so the output error is weighted (*Bryson's rule*);
3. the actuator limits are per wheel, so the amplitude constraint goes through the mecanum kinematics;
4. two of the constraints are on outputs, so they are soft, with a slack variable, exactly as 5.4.1 says they usually must be.

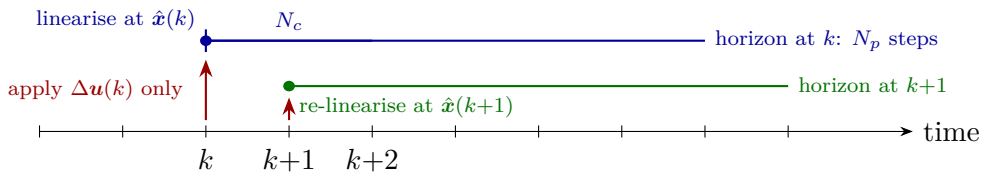
Everything else, the augmented model, the prediction matrices, the cost, the closed-form solution, the receding horizon, the constraint assembly and the quadratic programme, is the notes' material applied with $m = 3$ inputs and $p = 3$ outputs.

1 The big picture

The schematic on page 1 of Section 5 has a plant model and an optimiser inside the MPC box, a reference coming in, and a state estimator feeding estimated states back. Ours is that schematic with names filled in:

Schematic box	Course	Docking controller
Plant model	given (A, B, C), discrete LTI	kinematics of the chassis in the target frame, linearised each tick (DD 4.1–4.2)
Manipulated variables	$\boldsymbol{u}(k)$	body-velocity command $[u_b;\ v_b;\ r_b]$, converted to wheel speeds after the tick
Reference	y_r constant	docked pose $(0, 0, 0)$, or the standoff pose during the approach (DD 4.7)
State estimator	"estimated states"	the docking EKF, which supplies the pose and the velocity (DD 3)
Optimiser	`quadprog`	`quadprog`, same call, one extra variable for the slack

A picture to keep. The receding horizon of 5.2 and 5.3.5, with one addition: the dot at the start of each horizon is where the matrices are rebuilt. Everything inside a horizon is the course's linear MPC; what moves between horizons is the linearisation point.



2 Step 1: the plant model, and why it is rebuilt every tick

Course. 5.3.2 starts from a discrete linear time-invariant model

$$\boldsymbol{x}(k+1) = \boldsymbol{A}\boldsymbol{x}(k) + \boldsymbol{B}\boldsymbol{u}(k), \qquad \boldsymbol{y}(k) = \boldsymbol{C}\boldsymbol{x}(k), \tag{1}$$

with constant matrices, and remarks that $\boldsymbol{D}$ is dropped because the current input cannot affect the current output.

Ours. The thing being controlled is the pose $\boldsymbol{x}_c = [x;\ y;\ \theta]$ of the chassis in the target frame, driven by the body-velocity command. The continuous kinematics are nonlinear because the command is in body axes and the pose in target axes:

$$\dot{x} = u_b\cos\theta - v_b\sin\theta, \qquad \dot{y} = u_b\sin\theta + v_b\cos\theta, \qquad \dot{\theta} = r_b. \tag{2}$$

Two course tools turn this into the form 5.3.2 wants. Discretisation (2.5), Euler at the MPC period T_s:

$$\boldsymbol{x}_c(k+1) = \boldsymbol{f}_c(\boldsymbol{x}_c, \boldsymbol{u}) = \boldsymbol{x}_c + T_s \begin{bmatrix} u_b\cos\theta - v_b\sin\theta \\ u_b\sin\theta + v_b\cos\theta \\ r_b \end{bmatrix}. \tag{3}$$

Jacobian linearisation (6.2), but about the *current estimate and the last command* $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}) = (\hat{\boldsymbol{x}}_c(k), \boldsymbol{u}(k-1))$ instead of a fixed equilibrium:

$$\boldsymbol{A}_k = \left.\frac{\partial \boldsymbol{f}_c}{\partial \boldsymbol{x}_c}\right|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}} = \boldsymbol{I}_3 + T_s \begin{bmatrix} 0 & 0 & -(\bar{u}_b\sin\bar{\theta} + \bar{v}_b\cos\bar{\theta}) \\ 0 & 0 & \bar{u}_b\cos\bar{\theta} - \bar{v}_b\sin\bar{\theta} \\ 0 & 0 & 0 \end{bmatrix}, \tag{4}$$

$$\boldsymbol{B}_k = \left.\frac{\partial \boldsymbol{f}_c}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}} = T_s \begin{bmatrix} \cos\bar{\theta} & -\sin\bar{\theta} & 0 \\ \sin\bar{\theta} & \cos\bar{\theta} & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \boldsymbol{C} = \boldsymbol{I}_3. \tag{5}$$

The only nonlinear entries are the heading column of $\boldsymbol{A}_k$, which says how a heading change redirects the current command, and the rotation inside $\boldsymbol{B}_k$, which says which way in the target frame a body command pushes.

Why it differs. The course's constant $(\boldsymbol{A}, \boldsymbol{B})$ would be the linearisation at a single equilibrium, 6.2's $(\boldsymbol{x}^*, \boldsymbol{u}^*)$. The docked pose with zero command is such an equilibrium, and at it $\boldsymbol{B} = T_s\boldsymbol{I}$: the three axes decouple completely. That model is accurate only while the heading stays near zero. During the approach the heading can be 15° or more, where the fixed-point model mis-assigns a quarter ($\sin 15° = 0.26$) of a forward command to the lateral axis. Re-linearising at the current estimate every tick keeps the model accurate wherever the robot is, at the cost of rebuilding the matrices every tick. This is the same Taylor-series idea as 6.2 with a moving expansion point, and it is the one new idea in the whole controller.

The affine term. Linearising about a point that is not an equilibrium leaves a constant: $\boldsymbol{x}_c(k+1) \approx \boldsymbol{A}_k\boldsymbol{x}_c(k) + \boldsymbol{B}_k\boldsymbol{u}(k) + \boldsymbol{c}_k$ with $\boldsymbol{c}_k = \boldsymbol{f}_c(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}) - \boldsymbol{A}_k\bar{\boldsymbol{x}} - \boldsymbol{B}_k\bar{\boldsymbol{u}}$. The course never meets this because its model is linear. It disappears in Step 2.

Worked tick. With $T_s = 0.05$ s, at zero heading and a forward command of 0.2 m/s,

$$\boldsymbol{A}_k = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0.01 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \boldsymbol{B}_k = 0.05\,\boldsymbol{I}_3. \tag{6}$$

The entry $0.01 = T_s\bar{u}_b$ says: a heading error of one radian, held for one tick while driving forward at 0.2 m/s, moves the robot 10 mm sideways. At 15° heading with command [0.2; 0.05; 0] the same matrices are

$$\boldsymbol{A}_k = \begin{bmatrix} 1 & 0 & -0.0050 \\ 0 & 1 & 0.0090 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \boldsymbol{B}_k = \begin{bmatrix} 0.0483 & -0.0129 & 0 \\ 0.0129 & 0.0483 & 0 \\ 0 & 0 & 0.05 \end{bmatrix}, \tag{7}$$

where $\boldsymbol{B}_k$ is $0.05\,\boldsymbol{R}(15°)$ in its top-left block.

Check. `test_mpc_linearisation` differences your own $\boldsymbol{f}_c$ numerically and compares.

3 Step 2: the augmented incremental model

Course. 5.3.2 differences the state equation to get $\Delta\boldsymbol{x}(k+1) = \boldsymbol{A}\Delta\boldsymbol{x}(k) + \boldsymbol{B}\Delta\boldsymbol{u}(k)$, writes the output at the next step as $\boldsymbol{y}(k+1) = \boldsymbol{C}\boldsymbol{A}\Delta\boldsymbol{x}(k) + \boldsymbol{y}(k) + \boldsymbol{C}\boldsymbol{B}\Delta\boldsymbol{u}(k)$, and stacks the two into

$$\boldsymbol{x}_p(k) = \begin{bmatrix} \Delta\boldsymbol{x}(k) \\ \boldsymbol{y}(k) \end{bmatrix}, \qquad \boldsymbol{A}_p = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{0} \\ \boldsymbol{C}\boldsymbol{A} & \boldsymbol{I} \end{bmatrix}, \qquad \boldsymbol{B}_p = \begin{bmatrix} \boldsymbol{B} \\ \boldsymbol{C}\boldsymbol{B} \end{bmatrix}, \qquad \boldsymbol{C}_p = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} \end{bmatrix}. \tag{8}$$

The class exercise asks where the extra eigenvalues sit: the augmented matrix has the eigenvalues of $\boldsymbol{A}$ plus p eigenvalues at 1, the integrators that give the controller its integral action.

Ours. Identical, with $\boldsymbol{C} = \boldsymbol{I}_3$ so that $\boldsymbol{C}\boldsymbol{A} = \boldsymbol{A}_k$ and $\boldsymbol{C}\boldsymbol{B} = \boldsymbol{B}_k$:

$$\boldsymbol{x}_p(k) = \begin{bmatrix} \Delta\boldsymbol{x}_c(k) \\ \boldsymbol{x}_c(k) \end{bmatrix} \in \mathbb{R}^6, \qquad \boldsymbol{A}_p = \begin{bmatrix} \boldsymbol{A}_k & \boldsymbol{0}_{3\times3} \\ \boldsymbol{A}_k & \boldsymbol{I}_3 \end{bmatrix}, \qquad \boldsymbol{B}_p = \begin{bmatrix} \boldsymbol{B}_k \\ \boldsymbol{B}_k \end{bmatrix}, \qquad \boldsymbol{C}_p = \begin{bmatrix} \boldsymbol{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix}. \tag{9}$$

Since $\boldsymbol{A}_k$ is the identity plus terms of order T_s, all six eigenvalues of $\boldsymbol{A}_p$ sit at 1: the plant is three integrators (velocity command to position) and the augmentation adds three more. With $r_w = 0$ the solution would be the dead-beat one of Example 5.3, which here asks for velocity increments far beyond what the wheels can deliver; Step 4 shows the numbers.

Why the affine term vanishes. Differencing consecutive steps of $\boldsymbol{x}_c(k+1) = \boldsymbol{A}_k\boldsymbol{x}_c(k) + \boldsymbol{B}_k\boldsymbol{u}(k) + \boldsymbol{c}_k$, with the matrices frozen over the horizon, subtracts $\boldsymbol{c}_k$ from itself. So the incremental model is exactly 5.3.2's, and successive linearisation costs no extra term. This is the single most useful property of the course's $\Delta\boldsymbol{u}$ formulation for a nonlinear plant.

The initial increment. The course forms $\Delta\boldsymbol{x}(k) = \boldsymbol{x}(k) - \boldsymbol{x}(k-1)$ by differencing two measured states (Example 5.4 does exactly this by hand). Differencing two noisy pose estimates 50 ms apart would be a very poor velocity. We have an estimator that carries velocity as a state, so

$$\Delta\boldsymbol{x}_c(k) = T_s \begin{bmatrix} \hat{v}_x \\ \hat{v}_y \\ r_k \end{bmatrix}, \qquad \boldsymbol{x}_c(k) = \begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{\theta} \end{bmatrix}, \tag{10}$$

with the yaw rate straight from the gyro. Feeding the *measured* velocity here is also what gives the controller its disturbance rejection: if the wheels slip and deliver less than commanded, the increment state says so and the optimiser compensates.

Worked tick. From the zero-heading matrices above,

$$\boldsymbol{A}_p = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0.01 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0.01 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{array}\right], \qquad \boldsymbol{B}_p = \begin{bmatrix} 0.05\,\boldsymbol{I}_3 \\ 0.05\,\boldsymbol{I}_3 \end{bmatrix}. \tag{11}$$

4 Step 3: prediction over the horizon

Course. 5.3.3 stacks the predicted outputs $\boldsymbol{y}(k_i+1|k_i), \ldots, \boldsymbol{y}(k_i+N_p|k_i)$ into $\boldsymbol{Y}$ and the future increments $\Delta\boldsymbol{u}(k_i), \ldots, \Delta\boldsymbol{u}(k_i+N_c-1)$ into $\Delta\boldsymbol{U}$, and gets

$$\boldsymbol{Y} = \boldsymbol{F}\boldsymbol{x}_p(k_i) + \boldsymbol{\Phi}\Delta\boldsymbol{U}, \qquad \boldsymbol{F} = \begin{bmatrix} \boldsymbol{C}_p\boldsymbol{A}_p \\ \boldsymbol{C}_p\boldsymbol{A}_p^2 \\ \vdots \\ \boldsymbol{C}_p\boldsymbol{A}_p^{N_p} \end{bmatrix}, \qquad \boldsymbol{\Phi} = \begin{bmatrix} \boldsymbol{C}_p\boldsymbol{B}_p & \boldsymbol{0} & \cdots \\ \boldsymbol{C}_p\boldsymbol{A}_p\boldsymbol{B}_p & \boldsymbol{C}_p\boldsymbol{B}_p & \cdots \\ \vdots & \vdots & \ddots \\ \boldsymbol{C}_p\boldsymbol{A}_p^{N_p-1}\boldsymbol{B}_p & \boldsymbol{C}_p\boldsymbol{A}_p^{N_p-2}\boldsymbol{B}_p & \cdots \end{bmatrix}, \tag{12}$$

with the last column of $\boldsymbol{\Phi}$ stopping at $\boldsymbol{C}_p\boldsymbol{A}_p^{N_p-N_c}\boldsymbol{B}_p$ because increments beyond N_c are zero. `mpcgain.m` builds exactly these for a scalar system.

Ours. The same two matrices, with every scalar entry replaced by a 3×3 block: $\boldsymbol{F}$ is $3N_p \times 6$ and $\boldsymbol{\Phi}$ is $3N_p \times 3N_c$. The notes say in 5.3.4 that the MIMO extension "is straightforward following similar steps", and it is: the block in row i, column j of $\boldsymbol{\Phi}$ is $\boldsymbol{C}_p\boldsymbol{A}_p^{i-j}\boldsymbol{B}_p$ for $i \geq j$ and zero above the diagonal.

Why it differs. Nothing differs except dimension, and one consequence of Step 1: because $\boldsymbol{A}_p, \boldsymbol{B}_p$ change every tick, $\boldsymbol{F}$ and $\boldsymbol{\Phi}$ are rebuilt every tick. For $N_p = 40$, $N_c = 10$ that is a few thousand multiplications, well inside the 50 ms period.

Worked tick, with $N_p = 4$, $N_c = 2$ so the matrices fit on the page. At zero heading the three axes decouple and the x-axis rows of $\boldsymbol{F}$ and $\boldsymbol{\Phi}$, picked out with their x columns, are

$$\boldsymbol{F}_x = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 4 & 1 \end{bmatrix}, \qquad \boldsymbol{\Phi}_x = \begin{bmatrix} 0.05 & 0 \\ 0.10 & 0.05 \\ 0.15 & 0.10 \\ 0.20 & 0.15 \end{bmatrix}. \tag{13}$$

Compare Example 5.2: there $a = 0.8$, $b = 0.1$ gave a first column of $\boldsymbol{F}$ that saturated $(0.8, 1.44, 1.95, \ldots)$ and a $\boldsymbol{\Phi}$ whose entries grew by shrinking amounts. Here $a = 1$ and $b = T_s$, an integrator, so the first column of $\boldsymbol{F}_x$ counts steps and $\boldsymbol{\Phi}_x$ grows linearly: a velocity increment applied now moves the position by T_s more at every later step. The full 12×6 matrices only add the 0.01 coupling from heading into y (entries $0.03, 0.06, 0.10$ in $\boldsymbol{F}$ and $0.0005, 0.0015, 0.003$ in $\boldsymbol{\Phi}$), which is the heading column of $\boldsymbol{A}_k$ propagating.

Check. `test_mpc_prediction` compares your $\boldsymbol{F}, \boldsymbol{\Phi}$ against brute-force stepping of the augmented model, and the x-axis sub-blocks against `mpcgain.m` itself, which must agree to machine precision.

5 Step 4: the cost function and the unconstrained solution

Course. 5.3.4 writes

$$J = (\boldsymbol{Y}_r - \boldsymbol{Y})^{\mathsf{T}}(\boldsymbol{Y}_r - \boldsymbol{Y}) + \Delta\boldsymbol{U}^{\mathsf{T}}\bar{\boldsymbol{R}}\Delta\boldsymbol{U}, \qquad \bar{\boldsymbol{R}} = r_w\boldsymbol{I}, \tag{14}$$

with $\boldsymbol{Y}_r$ the set-point repeated N_p times, substitutes the prediction, differentiates, and finds

$$\Delta \boldsymbol{U} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi} + \bar{\boldsymbol{R}})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{Y}_r - \boldsymbol{F}\boldsymbol{x}_p(k_i)). \tag{15}$$

Example 5.3 then shows $r_w = 0$ giving a dead-beat answer and $r_w = 10$ a sluggish one.

Ours. Two weighting matrices appear:

$$J = (\boldsymbol{Y}_r - \boldsymbol{Y})^{\mathsf{T}}\bar{\boldsymbol{Q}}(\boldsymbol{Y}_r - \boldsymbol{Y}) + \Delta \boldsymbol{U}^{\mathsf{T}}\bar{\boldsymbol{R}}\Delta \boldsymbol{U}, \qquad \bar{\boldsymbol{Q}} = \boldsymbol{I}_{N_p} \otimes \boldsymbol{Q}_y, \quad \bar{\boldsymbol{R}} = r_w\, \boldsymbol{I}_{N_c} \otimes \boldsymbol{R}_u, \tag{16}$$

and the same derivative gives the same solution with $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ replaced by $\boldsymbol{\Phi}^{\mathsf{T}}\bar{\boldsymbol{Q}}\boldsymbol{\Phi}$ and $\boldsymbol{\Phi}^{\mathsf{T}}$ by $\boldsymbol{\Phi}^{\mathsf{T}}\bar{\boldsymbol{Q}}$:

$$\Delta \boldsymbol{U}^{\star} = (\boldsymbol{\Phi}^{\mathsf{T}}\bar{\boldsymbol{Q}}\boldsymbol{\Phi} + \bar{\boldsymbol{R}})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\bar{\boldsymbol{Q}}(\boldsymbol{Y}_r - \boldsymbol{F}\boldsymbol{x}_p(k)). \tag{17}$$

The weights come from Bryson's rule in 3.4,

$$\boldsymbol{Q}_y = \mathrm{diag}(1/y_{\mathrm{tol}}^2,\ 1/y_{\mathrm{tol}}^2,\ 1/\theta_{\mathrm{tgt}}^2), \qquad \boldsymbol{R}_u = \mathrm{diag}(1/\Delta u_{\mathrm{max},1}^2,\ 1/\Delta u_{\mathrm{max},2}^2,\ 1/\Delta u_{\mathrm{max},3}^2), \tag{18}$$

so that a position error equal to the coupler tolerance, a heading error equal to the internal heading target, and an increment equal to its limit each cost one unit. The scalar r_w is then the course's knob, scaling the whole increment term.

Why it differs. The course's unit weight on the output error is fine for one output. Ours are two metres and one angle in radians, and the acceptable errors are 80 mm and $1.5° = 0.026$ rad at full scale. With unit weights the controller would spend effort on heading only once the heading error was comparable in raw number to a position error, which is physically meaningless. Bryson's rule normalises each error by what counts as "one unit of bad". The kronecker product $\boldsymbol{I}_{N_p} \otimes \boldsymbol{Q}_y$ just repeats the 3×3 weight down the diagonal, once per predicted step, exactly as $\boldsymbol{Y}_r$ repeats the set-point.

Worked tick. Full-scale weights, $r_w = 1$:

$$\boldsymbol{Q}_y = \mathrm{diag}(156,\ 156,\ 1459), \qquad \boldsymbol{R}_u = \mathrm{diag}(4444,\ 4444,\ 1600). \tag{19}$$

Take the robot one metre out, 5 cm to the left, heading zero, moving forward at 0.2 m/s, so $\boldsymbol{x}_p = [0.01;\ 0;\ 0;\ -1;\ 0.05;\ 0]$, reference zero. The unconstrained solution over $N_c = 2$ is

$$\Delta \boldsymbol{U}^{\star} = \begin{bmatrix} 0.0170 & -0.0009 & -0.0000 \\ 0.0102 & -0.0005 & -0.0000 \end{bmatrix} \text{ (one row per step)}, \qquad \boldsymbol{u}(k) = \boldsymbol{u}(k{-}1) + \Delta \boldsymbol{u}(k) = \begin{bmatrix} 0.217 \\ -0.0009 \\ -0.0000 \end{bmatrix}, \tag{20}$$

and the predicted x over the four steps is $-0.989, -0.978, -0.966, -0.955$ m. Even this gentle tick asks for a forward increment of 0.017 m/s, above the acceleration limit of $T_s a_{\max} = 0.015$ m/s per tick. With the position weights on the internal target instead, nine times heavier ($\boldsymbol{Q}_y = \mathrm{diag}(1372, 1372, 1459)$), the same tick asks for 0.145 m/s, ten times the limit, and a command of 0.345 m/s above the speed cap: the dead-beat tendency of Example 5.3 with $r_w = 0$. Step 6 is what keeps either inside the actuator.

Why the position weight uses the tolerance and the heading weight the internal target. The obvious choice is one level for all three outputs, and it was the first choice. It fails for a reason that only the closed loop shows. The heading column of $\boldsymbol{A}_k$ makes the heading a lateral actuator through the forward speed (one radian of heading at 0.3 m/s is 0.3 m/s of lateral drift), a double-integrator path that runs through the wheel lag the kinematic model does not contain. With the lateral error weighted on the internal target the optimiser steers with the heading: on the demonstrator the heading swung by $\pm 20°$ during the approach and the yaw-rate command alternated at its increment limit. With the lateral error weighted on the tolerance it crabs instead, which is what a mecanum chassis is for. Accuracy at the dock does not suffer, because the steady-state error comes from the integral action of Step 2 and not from the weight: the lateral error at docking stays below 4 mm on both variants in the closed-loop tests. DD 4.5.

Check. `test_mpc_unconstrained` removes the limits and compares your `mpc_step` with this closed form.

6 Step 5: receding horizon and the explicit gains

Course. 5.3.5 applies only the first element of $\Delta \boldsymbol{U}$, moves one step, measures, and solves again. 5.3.6 pulls the first row out of the closed form to write $\Delta \boldsymbol{u}(k_i) = \boldsymbol{K}_y y_r - \boldsymbol{K}_{\text{mpc}} \boldsymbol{x}_p(k_i)$ and draws the block diagram with the integrator $z/(z-1)$ and the differencer $(z-1)/z$.

Ours. Identical, with the first *block* of three entries applied, as the notes say for MIMO. The gains $\boldsymbol{K}_y$ and $\boldsymbol{K}_{\text{mpc}} = [\boldsymbol{K}_x \;\; \boldsymbol{K}_y]$ exist in the same way, as the first three rows of $(\boldsymbol{\Phi}^{\mathsf{T}} \bar{\boldsymbol{Q}} \boldsymbol{\Phi} + \bar{\boldsymbol{R}})^{-1} \boldsymbol{\Phi}^{\mathsf{T}} \bar{\boldsymbol{Q}}(\cdot)$, but they change every tick with $\boldsymbol{F}, \boldsymbol{\Phi}$.

Why it differs. Only Step 1 again: the gains are time-varying because the linearisation is. They still matter for two reasons. They are the fallback if a solver is ever unavailable, a 3×6 gain times the state followed by saturation. And near the dock, where the heading stays near zero and the matrices stop changing, they settle to constants, so the final approach is in effect the course's LTI controller, which is a useful thing to say in the report.

7 Step 6: constraints

Course. 5.4.1 lists three types: (i) on the increments $\Delta u_{\min} \le \Delta \boldsymbol{u} \le \Delta u_{\max}$, (ii) on the amplitudes $u_{\min} \le \boldsymbol{u} \le u_{\max}$ through the accumulation $\boldsymbol{U} = \boldsymbol{C}_1 \boldsymbol{u}(k_i - 1) + \boldsymbol{C}_2 \Delta \boldsymbol{U}$, and (iii) on outputs $y_{\min} \le \boldsymbol{y} \le y_{\max}$ through the prediction, with the warning that output constraints may be infeasible and are usually made soft with a slack $s_v \ge 0$. 5.4.2 rewrites each two-sided limit as two one-sided rows and stacks them into $\boldsymbol{M} \Delta \boldsymbol{U} \le \boldsymbol{\gamma}$ with $4mN_c + 2pN_p$ rows.

Ours. Every one of the three types is used, and the mapping is:

Course type	Physical limit here	Rows	Hard or soft
(i) increments	acceleration limit (anti-slip): $\lvert \Delta \boldsymbol{u} \rvert \le T_s [a_{\max};\ a_{\max};\ \alpha_{\max}]$	$6N_c$	hard
(ii) amplitudes	speed cap $\lvert u_b \rvert + \lvert v_b \rvert \le v_{\max}$ (four facets), lowered in the final approach, and $\lvert r_b \rvert \le r_{\max}$	$6N_c$	hard
(ii) through a map	wheel speeds $\lvert \boldsymbol{J}^{-} \boldsymbol{u} \rvert \le \omega_{\max}$, four wheels	$8N_c$	hard
(iii) output	no penetration past the contact band, $x(k+j) \le x_{\max}$	N_p	soft
(iii) output	heading corridor $\lvert \theta(k+j) \rvert \le \theta_{\max}$, marker stays in view	$2N_p$	soft

The accumulation matrices are the course's, block form: $\boldsymbol{C}_1 = \boldsymbol{1}_{N_c} \otimes \boldsymbol{I}_3$, $\boldsymbol{C}_2 = \boldsymbol{L}_{N_c} \otimes \boldsymbol{I}_3$ with $\boldsymbol{L}$ lower-triangular ones.

Why it differs. Four things to explain.

The shape of the speed cap. The notes' amplitude limit is a box, $u_{\min} \le \boldsymbol{u} \le u_{\max}$ on each input. A box on the two body velocities is 41 % faster along its diagonal than along its axes, and the optimiser finds this: on the demonstrator it yawed the body to the edge of the heading corridor for the whole approach, to drive on the diagonal. The cap is therefore the L1 bound $\lvert u_b \rvert + \lvert v_b \rvert \le v_{\max}$, written as its four facets $\pm u_b \pm v_b \le v_{\max}$, four rows per step. It bounds the speed norm in every direction, and it is the shape of the wheel-speed limit at zero yaw rate, $\lvert u_b \rvert + \lvert v_b \rvert + L \lvert r_b \rvert \le R_w \omega_{\max}$, so the chassis is fastest along its own axes and has no reason to yaw. The yaw rate keeps its two box rows. Still type (ii), still linear in $\Delta \boldsymbol{U}$ through $\boldsymbol{C}_1$ and $\boldsymbol{C}_2$, still six rows per step.

The wheel rows. The notes only have box limits on $\boldsymbol{u}$. Our actuators are four motors, each with a speed limit, and the mecanum inverse kinematics $\boldsymbol{\omega} = \boldsymbol{J}^{-} \boldsymbol{u}$ is a constant linear map, so the wheel limits are box limits on $\boldsymbol{J}^{-} \boldsymbol{U}$: still linear in $\Delta \boldsymbol{U}$, still type (ii), eight rows per step instead of six. This is how the kinematics enter the controller. The decision variables stay in body velocity because the fourth wheel degree of freedom only scrubs the rollers and the kinematic model cannot see it.

Why the output rows are soft, and how the slack works. 5.4.1 says output constraints "may violate constraints of types (i) and (ii)" and "can make the optimisation problem infeasible". That is not

hypothetical here. At contact the estimate can sit a few millimetres past the plane, through wheel lag or estimation noise, and no increment sequence inside the acceleration limit can bring the predicted x back below the plane at the first steps, so a hard row has no feasible point and `quadprog` returns nothing. The remedy is the notes' s_v: append one slack $s \geq 0$ to the decision vector, subtract it from every soft row so that $x(k+j) - s \leq x_{\max}$, and penalise it in the cost. In the QP this means

$$\boldsymbol{z} = \begin{bmatrix} \Delta \boldsymbol{U} \\ s \end{bmatrix}, \qquad \boldsymbol{E}_{\text{ext}} = \begin{bmatrix} \boldsymbol{E} & \boldsymbol{0} \\ \boldsymbol{0}^{\mathsf{T}} & \epsilon \end{bmatrix}, \qquad \boldsymbol{f}_{\text{ext}} = \begin{bmatrix} \boldsymbol{f} \\ \rho_s \end{bmatrix}, \qquad \boldsymbol{M}_{\text{ext}} = \begin{bmatrix} \boldsymbol{M}_{\text{hard}} & \boldsymbol{0} \\ \boldsymbol{M}_{\text{soft}} & -\boldsymbol{1} \end{bmatrix}, \tag{21}$$

with the lower bound $s \geq 0$ passed to the solver. The penalty is *linear*, $\rho_s s$, on purpose: a linear penalty larger than any constraint multiplier is exact, meaning s is zero whenever the hard constraint can be met, whereas a quadratic penalty always trades a small violation against tracking. Because the multipliers scale with $\boldsymbol{Q}_y$, so must ρ_s; the parameter file sets it to $100\, N_p \max(\boldsymbol{Q}_y)$.

Why a contact band. With the plane at first contact, the controller bounces: the lag carries the robot a millimetre past the plane, the exact penalty demands maximum reverse, the tracking cost drives it forward again, and the dock criterion never sees a slow robot. A coupler needs a few millimetres of engagement travel anyway, so the constraint plane sits at the end of that travel, $x_{\max} = 10$ mm, and the robot settles into the band instead of fighting its edge.

Assembly. Row count $12N_c + 8N_c + 3N_p = 20N_c + 3N_p$, 320 rows for $N_c = 10$, $N_p = 40$, against 30 decision variables plus the slack. Unlike the notes' LTI case, the output blocks of $\boldsymbol{M}$ change every tick because $\boldsymbol{\Phi}$ does; the increment and amplitude blocks are constant.

Worked tick, continued. The unconstrained forward increment of 0.017 m/s violates (i), whose limit is $T_s a_{\max} = 0.015$ m/s; the command 0.217 m/s is inside the cap of (ii). Solving the same tick with the constraints gives

$$\Delta \boldsymbol{U}^{\star} = \begin{bmatrix} 0.0150 & -0.0009 & -0.0000 \\ 0.0102 & -0.0005 & -0.0000 \end{bmatrix}, \qquad \boldsymbol{u}(k) = \begin{bmatrix} 0.215 \\ -0.0009 \\ -0.0000 \end{bmatrix}, \tag{22}$$

with the forward increment row active at the first step only, and the lateral and yaw columns unchanged because the axes decouple at zero heading. The predicted x is $-0.989, -0.978, -0.967, -0.956$ m. With the heavier internal-target weights of the aside in Step 4 the same machinery clamps 0.145 to 0.015 at both steps and the command to 0.215 m/s instead of 0.345: the first step then advances 10.8 mm instead of 17 mm, and the cap of (ii) would bind about six steps on. The slack is zero in both cases: no soft row is active this far from the target.

Check. `test_mpc_constraints` verifies every limit over the horizon from a state where the amplitude limit must bind, and the row count.

8 Step 7: the quadratic programme

Course. 5.4.2 drops the constant term and writes $J = \frac{1}{2}\Delta \boldsymbol{U}^{\mathsf{T}} \boldsymbol{E} \Delta \boldsymbol{U} + \Delta \boldsymbol{U}^{\mathsf{T}} \boldsymbol{f}$ with $\boldsymbol{E} = 2(\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi} + \bar{\boldsymbol{R}})$, $\boldsymbol{f} = -2\boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{Y}_r - \boldsymbol{F}\boldsymbol{x}_p)$, then gives the correspondence table to `quadprog`$(\boldsymbol{H}, \boldsymbol{f}, \boldsymbol{A}, \boldsymbol{b})$.

Ours. The same table with three edits:

Course	`quadprog` argument	Docking controller
$\boldsymbol{E} = 2(\boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{\Phi} + \bar{\boldsymbol{R}})$	$\boldsymbol{H}$	$2(\boldsymbol{\Phi}^{\mathsf{T}} \bar{\boldsymbol{Q}} \boldsymbol{\Phi} + \bar{\boldsymbol{R}})$, bordered by ϵ for the slack; rebuilt each tick
$\boldsymbol{f} = -2\boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{Y}_r - \boldsymbol{F}\boldsymbol{x}_p)$	$\boldsymbol{f}$	$-2\boldsymbol{\Phi}^{\mathsf{T}} \bar{\boldsymbol{Q}}(\boldsymbol{Y}_r - \boldsymbol{F}\boldsymbol{x}_p)$, with ρ_s appended
$\boldsymbol{M}$	$\boldsymbol{A}$	$\boldsymbol{M}_{\text{ext}}$, output blocks rebuilt each tick
$\boldsymbol{\gamma}$	$\boldsymbol{b}$	rebuilt each tick, as in the notes
–	`lb`	$[-\infty;\ \ldots;\ 0]$, the slack is non-negative

One practical note the course does not need: the QP is solved with the `active-set` algorithm from a feasible starting point ($\Delta \boldsymbol{U} = \mathbf{0}$ and just enough slack), and retried with `interior-point-convex` if that reports failure. Each has a weakness the other does not. Interior-point occasionally reports the problem unbounded on the near-degenerate ticks at contact; active-set can stall on a degenerate vertex, such as driving at the vertex of the L1 cap with two wheels at their limit, where four constraints meet in three variables. Only both failing triggers the safe stop. One tick costs about 17 ms in MATLAB against the 50 ms period.

9 Step 8: around the optimiser, which the course leaves to a schematic

Three pieces of the loop are drawn but not developed in the notes.

The reference and the limits are staged. 5.3.4 assumes a constant set-point over the horizon, and we keep that, but a supervisor switches it: the standoff pose $(-0.3, 0, 0)$ with the approach speed limit until the robot is aligned, then the origin with the final speed limit. The limit is lowered at the increment rate rather than stepped, otherwise the amplitude rows become infeasible at the switch, because $\boldsymbol{u}(k-1)$ would already lie outside them.

The estimator closes the loop. The schematic's "state estimator" is the docking EKF. Its pose fills $\boldsymbol{y}(k)$ and its velocity fills $\Delta \boldsymbol{x}_c(k)$, which is where the course's separation principle (4.2) is being relied on in spirit: the controller is designed as if the estimate were the state. For a constrained MPC there is no formal separation theorem, which is why the integrated test runs both together.

The command leaves as wheel speeds. The last line of every tick is $\boldsymbol{\omega}_{\text{cmd}} = \boldsymbol{J}^{-}\boldsymbol{u}(k)$ to the four PID loops.

10 Everything that goes beyond the notes, on one page

Divergence	Course position	Docking controller and reason
Time-varying model	$(\boldsymbol{A}, \boldsymbol{B})$ constant	Jacobians of the kinematics at the current estimate every tick; the fixed-point model is wrong away from zero heading. DD 4.2.
Affine term	none, model is linear	cancels in the incremental form; stated and used. DD 4.2.
Initial increment	difference of two measured states	T_s times the EKF velocity; differencing noisy poses would be a poor velocity. DD 4.3.
Output weighting	unit weight, SISO	Bryson's rule from 3.4: positions on the coupler tolerance, heading on the internal target; mixed units, and the heading must not become a steering actuator. DD 4.5.
Increment weighting	$r_w \boldsymbol{I}$	Bryson on the increment limits times r_w; same knob, normalised units. DD 4.5.
Wheel limits	box limits on $\boldsymbol{u}$ only	box limits on $\boldsymbol{J}^{-}\boldsymbol{U}$, type (ii) through a constant linear map. DD 4.6.
Soft output rows	s_v mentioned, not formulated	one slack with a linear exact penalty, scaled with $\boldsymbol{Q}_y$; hard rows infeasible at contact. DD 4.6.
Contact band	–	constraint plane at the end of the coupler's engagement travel, not at first contact; avoids bouncing. DD 4.6.
Speed-cap shape	box on each input	L1 bound $\lvert u_b\rvert + \lvert v_b\rvert \le v_{\max}$, four facets per step; a box is 41 % faster on its diagonal and the optimiser yaws to use it. DD 4.6.
Heading corridor	–	output constraint keeping the marker in view and bounding how far the optimiser steers with the heading. DD 4.6.
Staged reference and limits	constant set-point	supervisor; limits lowered at the increment rate for feasibility. DD 4.7.
Solver	`quadprog` default	active-set with a feasible start; interior-point failed at contact. DD 4.7.
Lag model (optional)	–	six-state plant with the wheel time constant; removes the $v_{\text{final}}\tau$ overshoot. DD 4.8.

11 Map from the notes to the code and the tests

Function (yours)	Notes	Design document	Test
`mecanum_ik`, `mecanum_fk`	–	DD Appendix A	`test_mpc_kinematics`
`mpc_linearise`	2.5, 6.2	DD 4.1–4.2	`test_mpc_linearisation`
`mpc_predict_matrices`	5.3.2–5.3.3, `mpcgain.m`	DD 4.3–4.4	`test_mpc_prediction`
`mpc_constraints`	5.4.1–5.4.2	DD 4.6	`test_mpc_constraints`
`mpc_step`	5.3.4–5.3.6, 5.4.2	DD 4.5–4.7	`test_mpc_unconstrained`, `test_mpc_constraints`
whole loop	5.2, schematic in 5.1	DD 4.7, 5	`test_mpc_closed_loop`, `test_integrated_docking`